FoodRescue

Project Overview & Technical Guide

Purpose

A food donation prototype connecting donors who have surplus food with NGOs that can respond to pickup requests.

Frontend Next.js App Router · React · TypeScript	**Backend** Next.js route handlers · server-side validation
Data Prisma ORM · SQLite local database	**Assistant** Google Gemini Developer API

Prepared from the source code and configuration in this workspace. This document excludes secret values.

Security: Rotate any Gemini API key previously stored in the sample environment file. Keep replacement credentials only in a local ignored environment file.

1. Product and User Flows

FoodRescue supports three roles: donors, NGO operators, and administrators. The product is a prototype; listings and actions are not proof of real-world verification or completed pickups.

Donor flow

- Enter pickup location, region, food type, positive quantity/unit, and food expiry.
- Optionally use browser location; the server calls OpenStreetMap Nominatim reverse geocoding to suggest a readable address and region.
- Review NGO choices at **/select-ngo**, then broadcast the donation. Broadcasts expire at the earlier of one hour or the food expiry (as described in the chatbot system prompt).
- Donation validation requires expiry at least ten minutes in the future; supported units are kg, servings, boxes, and litres.

NGO flow

- Register or sign in at **/login**. NGO accounts must be active and approved to authenticate.
- View unexpired broadcasts for the assigned region; Unassigned NGOs can see requests across regions. Busy NGOs do not receive open requests.
- Accept a request, decline it individually, or move an accepted pickup through **in_progress** and **completed**. The dashboard refreshes requests periodically.

Administrator flow

- Sign in at **/admin/login**. The admin portal provides regional summaries and NGO account management.
- Review NGO accounts and manage region assignment/activation and approval state. Admin access is protected by a signed session cookie.

2. Frontend Architecture

Area	Routes / files	Responsibility
/ landing	app/page.tsx; components/hero-section.tsx	Donor-facing entry point and impact content.
NGO selection	app/select-ngo/page.tsx; components/ngo/*	Reads search parameters and builds NGO selection experience.
NGO portal	app/login/page.tsx; app/ngo/dashboard/page.tsx; components/ngo-dashboard.tsx	Registration/login, request feed and pickup status actions.
Admin portal	app/admin/login/page.tsx; app/admin/page.tsx; components/admin-dashboard.tsx	Admin authentication and operations dashboard.
Global UI	app/layout.tsx; components/site-header.tsx; components/help-chatbot.tsx	Shared layout, navigation and global chat widget.
Styling	app/globals.css; Tailwind CSS 4	Global theme and utility-based styling.

The app uses the Next.js App Router. Server components load protected pages/data; interactive widgets are client components. Alias **@/*** maps to the workspace root in tsconfig.json.

3. Backend and API

API endpoints are Next.js route handlers under app/api. They parse and validate input, authenticate protected requests, update records through Prisma, and return JSON responses.

API area	Routes	Main behavior
Donations	/api/donations; /api/donations/[id]; /api/donations/[id]/status	Create donation broadcasts; retrieve/update donation data and status.
NGO requests	/api/ngo-requests; /[id]/accept; /[id]/decline; /[id]/status	List eligible requests, accept/decline, advance pickup status. Requires NGO session.
NGO auth	/api/ngo-auth/register, /login, /logout, /availability	Create account, authenticate, end session, update availability.
Admin	/api/admin/dashboard; /api/admin/ngos; /api/admin/ngos/[id]	Dashboard summaries and NGO account management. Admin protected.
Admin auth	/api/admin-auth/login; /api/admin-auth/logout	Admin sign-in and sign-out.
Assistant	POST /api/help	Validates conversation, enforces in-memory rate limit, calls Gemini.
Geocoding	GET /api/location/reverse?lat=…&lng;=…	Reverse lookup using Nominatim; returns address and region.

Validation / concurrency: donation creation validates address, region, food type, quantity, units, expiry and coordinate ranges. NGO acceptance uses a conditional update to avoid accepting an already-claimed or expired request. Expired broadcasts are marked expired when request feeds are queried.

4. Database and Persistence

Prisma connects the application to SQLite through **DATABASE_URL**. The Prisma client singleton is in lib/prisma.ts; schema and migration SQL are under prisma/. The documented local database file is prisma/dev.db.

Model	Important fields / purpose
Donation	Pickup address and optional coordinates; food type, quantity and unit; expiry; status; selected NGO; region; timestamps. Indexed by status and createdAt.
NgoAccount	Name and unique normalized name key; scrypt password hash; region; active flag; availability; review status; creation date.
NgoRequestDecline	Donation-to-NGO decline record; unique pair prevents duplicate declines. Relations cascade on deletion.

Data lifecycle

- Donation is created with status **broadcasting**.
- An NGO can accept only if it is active/approved, available, within region (or Unassigned), and the donation is not expired or already accepted.
- Accepted status transitions to **in_progress**, then **completed**. Overdue broadcasts become **expired**.
- Declines are per NGO and do not hide the same request from every NGO.

Migration commands

npm run db:migrate applies Prisma development migrations. **npm run db:studio** opens Prisma Studio for local inspection. Migrations currently present are initial schema, NGO accounts, admin regions, availability/declines, and NGO review status.

5. Chatbot and External Integrations

The floating assistant UI is components/help-chatbot.tsx. It posts the recent conversation to **POST /api/help**; the server uses **@google/genai** and GoogleGenAI to call the Gemini Developer API. The key is server-only and must not use a NEXT_PUBLIC_ prefix.

- Model selection uses GEMINI_MODEL when present; otherwise the route falls back to **gemini-3.8-flash** in the current source.
- The route supplies a FoodRescue-specific system instruction, sends up to 12 messages, truncates message text, sets a maximum output token count, and caps requests at 15 per minute per forwarded client IP using in-memory state.
- The assistant has no live web-search grounding in this configuration; answers may be stale. The client shows an error returned by the server when provider calls fail.
- Reverse geocoding uses the public OpenStreetMap Nominatim endpoint with a User-Agent and an eight-second timeout.
- Vercel Analytics is rendered only when NODE_ENV is production.

6. Authentication and Security

Control	Implementation
NGO passwords	Node crypto scrypt with per-password random salt; constant-time comparison.
NGO session	HMAC-SHA256 signed payload, HTTP-only cookie, seven-day lifetime; active and approved account rechecked against database.
Admin session	Environment-based username/password; HMAC-SHA256 signed cookie, HTTP-only, SameSite strict, eight-hour lifetime.
Environment files	.env and .env*.local are ignored by Git. Keep credentials there; share only placeholders.
Production	Set a unique NGO_SESSION_SECRET of at least 32 random bytes and strong unique admin credentials. Use HTTPS and verify deployment secret management.

Secret incident note: A real-looking Gemini key was found in .env.example during documentation generation and has been replaced with a placeholder. Revoke/rotate that key in Google AI Studio and review repository history if it was committed. This report does not print or embed the credential.

7. Environment and Local Setup

Variable	Purpose
DATABASE_URL	Prisma connection string; sample SQLite value is file:./dev.db.
NGO_SESSION_SECRET	Signs NGO and admin session payloads.
ADMIN_USERNAME / ADMIN_PASSWORD	Credentials for the administrator portal.
GEMINI_API_KEY	Private server-side Google Gemini API key.
GEMINI_MODEL	Optional model override; current documented default is gemini-3.8-flash.

Typical local workflow

- Install dependencies using the package manager used by the project (npm install; package metadata also identifies pnpm).
- Copy the safe template .env.example to .env and set local values. Never commit real credentials.
- Run **npm run db:migrate**.
- Start with **npm run dev**; open the local URL printed by Next.js.
- Production: build with **npm run build** and start with **npm start**.

8. Known Limitations and Notes

- Prototype only: sample NGO entries/actions are not independently verified; no real pickup or food safety guarantee is provided.
- SQLite and local dev database are configured; a production database/deployment topology is not specified in the reviewed project files.
- The chatbot rate-limit map is process-local and resets on restart; distributed production deployments need shared rate limiting.
- No live search for chatbot; validate time-sensitive answers independently.
- next.config.mjs sets typescript.ignoreBuildErrors=true; remove or address this before relying on production build checks.

- Review API access controls and data minimization before deployment, especially donation addresses and NGO personal data.

Source of truth: project source, Prisma schema/migrations, package.json and README.md in this workspace. Generated 2026-10-09.